A Recursive CTE is a CTE that refers to itself to repeatedly generate or retrieve rows until a termination condition is met.

It is particularly useful when data has a hierarchical or parent-child relationship.

For example:

- Employee-manager hierarchy.
- Organization structure.
- Folder and subfolder hierarchy.
- Category and subcategory hierarchy.
- Generating number sequences.
- Finding all descendants of a particular node.

Let's understand it step by step.

1. Basic structure

A recursive CTE has three important parts:

```
WITH cte_name AS (


    -- 1. Anchor query

    SELECT ...


    UNION ALL


    -- 2. Recursive query

    SELECT ...

    FROM cte_name

    WHERE termination_condition
```

```
)

-- 3. Final query
SELECT *
FROM cte_name;
```

The two important components:

1. Anchor member: Executes first and provides the initial rows.
2. Recursive member: Uses the previous iteration's results to generate the next set of rows.

The recursion stops when the recursive query produces no more rows.

2. Example: Generate numbers from 1 to 5

Let's start with the simplest example.

```
WITH numbers (n) AS (

    -- Anchor member
    SELECT 1
    FROM dual

    UNION ALL

    -- Recursive member
    SELECT n + 1
```

```
    FROM numbers
    WHERE n < 5

)
SELECT n
FROM numbers;
```

Result:

Iteration	Input	Calculation	Output
Anchor	–	Initial value	1
1	1	1 + 1	2
2	2	2 + 1	3
3	3	3 + 1	4
4	4	4 + 1	5
5	5	Condition 5 < 5 is false	Stops

Visual execution

Anchor query

1

n + 1

Recursive result

2

Recursive result

3

Recursive result

4

Recursive result

5

n < 5 is false → Stop

The important point is that the recursive query reads the rows produced by the previous iteration.

3. Practical example: Employee-manager hierarchy

Consider this employee table:

Employee ID	Employee	Manager ID

1	Amit	NULL
2	Rahul	1
3	Priya	1
4	Neha	2
5	Ravi	2
6	Suman	3

The hierarchy looks like this:

Amit (1)

Rahul (2)

Neha (4)

Ravi (5)

Priya (3)

Suman (6)

Requirement: Find all employees reporting directly or indirectly to Amit.

SQL query (Oracle)

```
WITH employee_hierarchy (emp_id, employee, manager_id, level_no) AS (
```

```
    -- Anchor: Find the top-level manager
    SELECT
        emp_id,
        employee,
        manager_id,
        1
    FROM employees
    WHERE emp_id = 1

    UNION ALL

    -- Recursive member: Find employees reporting to previous level
    SELECT
        e.emp_id,
        e.employee,
        e.manager_id,
        h.level_no + 1
    FROM employees e
    JOIN employee_hierarchy h
        ON e.manager_id = h.emp_id

)
SELECT *
```

```
FROM employee_hierarchy

ORDER BY level_no, emp_id;
```

Result:

Employee	Manager ID	Level
Amit	NULL	1
Rahul	1	2
Priya	1	2
Neha	2	3
Ravi	2	3
Suman	3	3

Notice the recursive join:

```
ON e.manager_id = h.emp_id
```

This is the key to understanding the hierarchy.

- First, we find Amit.
- Next, we find employees whose manager_id is Amit's ID.
- Then, we find employees whose manager_id matches the IDs found in the previous iteration.
- This continues until no more employees are found.

4. How recursion works internally

Let's trace the employee example.

Iteration	Previously found employee	Next employees found
Anchor	—	Amit (1)
1	Amit (1)	Rahul (2), Priya (3)
2	Rahul (2), Priya (3)	Neha (4), Ravi (5), Suman (6)
3	Neha, Ravi, Suman	No more employees

The recursive query does not restart from the entire employee table as its input. It uses the previous iteration's generated rows, joining them back to the original employee table.

5. Recursive CTE vs normal CTE

Normal CTE	Recursive CTE
Does not reference itself	References itself
Usually executes as a single logical query	Repeatedly generates rows

Used to simplify complex queries	Used for hierarchical and iterative problems
No anchor/recursive distinction	Has anchor and recursive members

6. Important Oracle syntax note

Oracle supports recursive subquery factoring from Oracle 11g Release 2 onward.

There are two details to remember:

- Oracle requires the recursive member to use UNION ALL and reference the CTE itself.
- Unlike some other databases, Oracle requires a CYCLE clause when you need explicit cycle detection in a recursive CTE. Without cycle protection, cyclic hierarchical data can cause an error.

Example of cycle detection:

```
WITH employee_hierarchy (emp_id, employee, manager_id, level_no) AS (

    SELECT emp_id, employee, manager_id, 1

    FROM employees

    WHERE emp_id = 1



    UNION ALL



    SELECT e.emp_id, e.employee, e.manager_id, h.level_no + 1

    FROM employees e
```

```
    JOIN employee_hierarchy h
      ON e.manager_id = h.emp_id
)
CYCLE emp_id SET is_cycle TO 'Y' DEFAULT 'N'
SELECT *
FROM employee_hierarchy;
```

Oracle also has a traditional hierarchical query feature using START WITH ... CONNECT BY PRIOR, which is often simpler for Oracle-specific hierarchy queries.

7. Test your understanding

Suppose the employee hierarchy is:

- A is the manager of B and C.
- B is the manager of D and E.
- C is the manager of F.

Question: If the anchor query starts with employee A, how many recursive iterations are needed before the query stops?

2 iterations

3 iterations

4 iterations

Check answer

One important distinction: The anchor query is not counted as a recursive iteration. It establishes the starting point.